Chapter 2: Processes and Threads

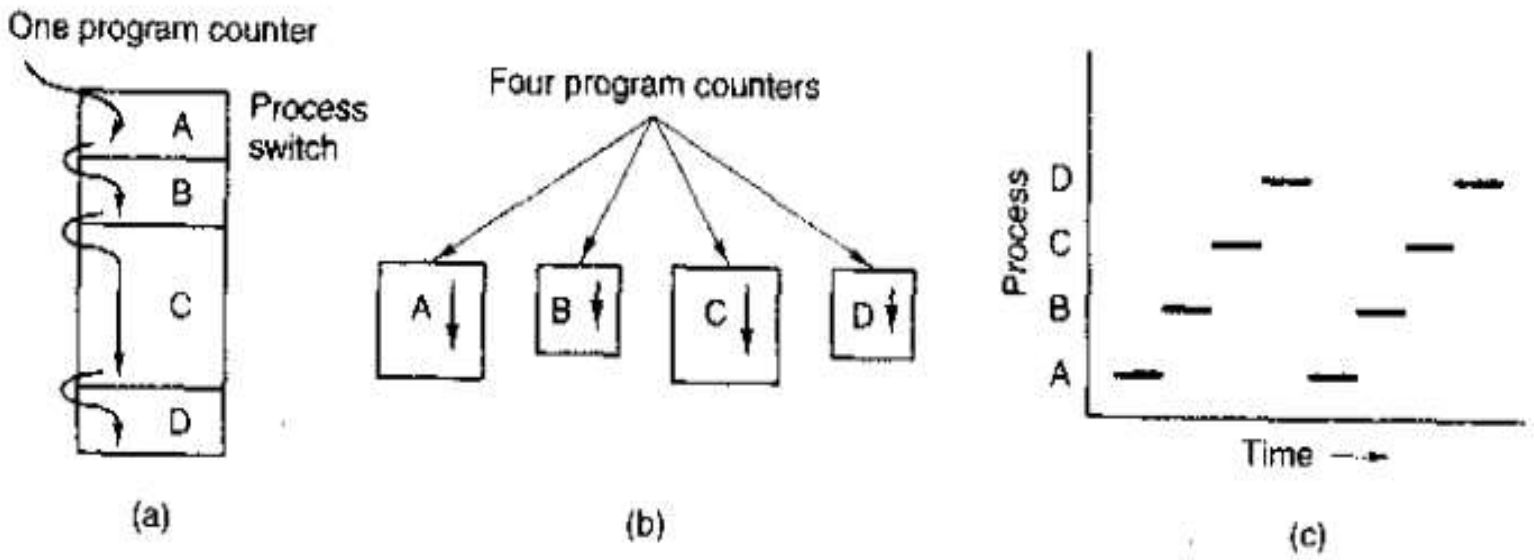
- PROCESS CONCEPT
- PROCESS STATES AND TRANSITIONS
- OPERATIONS ON PROCESSES
- IMPLEMENTATION OF PROCESSES
- COOPERATING PROCESSES
- THREADS

Process Concept

- An operating system executes a variety of programs:
  - Batch system – jobs
  - Time-shared systems – user programs or tasks
- Textbook uses the terms job and process almost interchangeably.
- **Process** – a program in execution
- Process execution must progress in **sequential fashion**.
  - Illusion of parallelism (pseudo parallelism)
  - True parallelism in multi-processor
- Therefore conceptual model has been evolved over time

Process concept…

A. Multi-programming of four programs
B. Conceptual model of four independent, Sequential processes.
C. only one program is active at once.



Process concept…

- The rate at which process perform it's computation is not uniform
- Thus processes must not be programmed with built-in assumption about timing.
  - Consider….. How Streamer tape restore a backup file
- The difference b/n Program and Process is subtle but crucial.
  - Consider…. How people live the way they live
- The key idea is a process is an activity of some kind.
- It has
  - A program
  - Input/output
  - State

Process Creation

- OS needs some way to make sure all necessary processes exist.
- Not an issue for simple systems like Microwave oven.(simple Embedded).
- There are four principal events that cause the process to be created

- System initialization
  - Foreground processes
  - Background processes (daemons)

  UNIX- *ps program ; Windows CTRL+ALT+DEL*
- Execution of a process system call by a running process
  - Several related but otherwise independent interacting processes

  e.g. fetching of large amount of data on the network
- A user request to create a process
  - Different kinds of process creation in different system e.g. UNIX and Windows
  - Possible to create in an interactive environment
- Initiation of a batch job
  - Applies only on large main frames
  - Users can submit batches jobs (possibly remotely)

Process creation…

- Typically in all cases…
  - Existing process —executes→ process Creation system call
    - User process
    - System process invoked from I/O
    - A batch manager process
    - Tells OS to create New Process and indicate, directly or indirectly, Which program to run on it

Process creation…

- In Unix, there is only one system call to create a new process called: ***fork***
- This call creates an exact clone of the calling process.
- The two process have parent and child relationship

  i.e. they share

  - Same memory image
  - Same environmental strings
  - Same open file

- Usually the child executes <u>*execve*</u>

  To change the memory image and run a new program

Accomplish redirection of
standard Input
standard output
standard error

Process Creation…

- But in Windows
  - A single win32 function call→ create process
  - Create process handles both
    - Process creation
    - Loading the correct program

Process Creation…

- Parent process create children processes which, in turn, create other processes forming a tree of processes.
- Each process is assigned a unique process identifier (pid)
- Win32 has about 100 other functions for managing and synchronizing processes and related issues.
  - Address space- both parent and child have distinct address space.
  - Resource sharing
    - Parent and children share all resources.
    - Children share subset of parent's resources.
    - Parent and child share no resources.
  - Execution
    - Parent and children execute concurrently.
    - Parent waits until some or all of its children terminate.

Process Termination

- A process terminates when it finishes executing its final statement and asks the operating system to delete it by using the exit() system call.
- Termination can occur in other circumstances as well.
  - Normal exit (Voluntary)
    - When a compiler finishes its work it executes a system call
      - Exit→ Unix
      - ExitProcess →Windows
    - Screen oriented programs support Voluntary exit
  - Error exit (Voluntary)
    - If a user writes a command cc foo.c and if no file exist, then the compiler simply exits.
    - Screen oriented systems gives u a second chance.

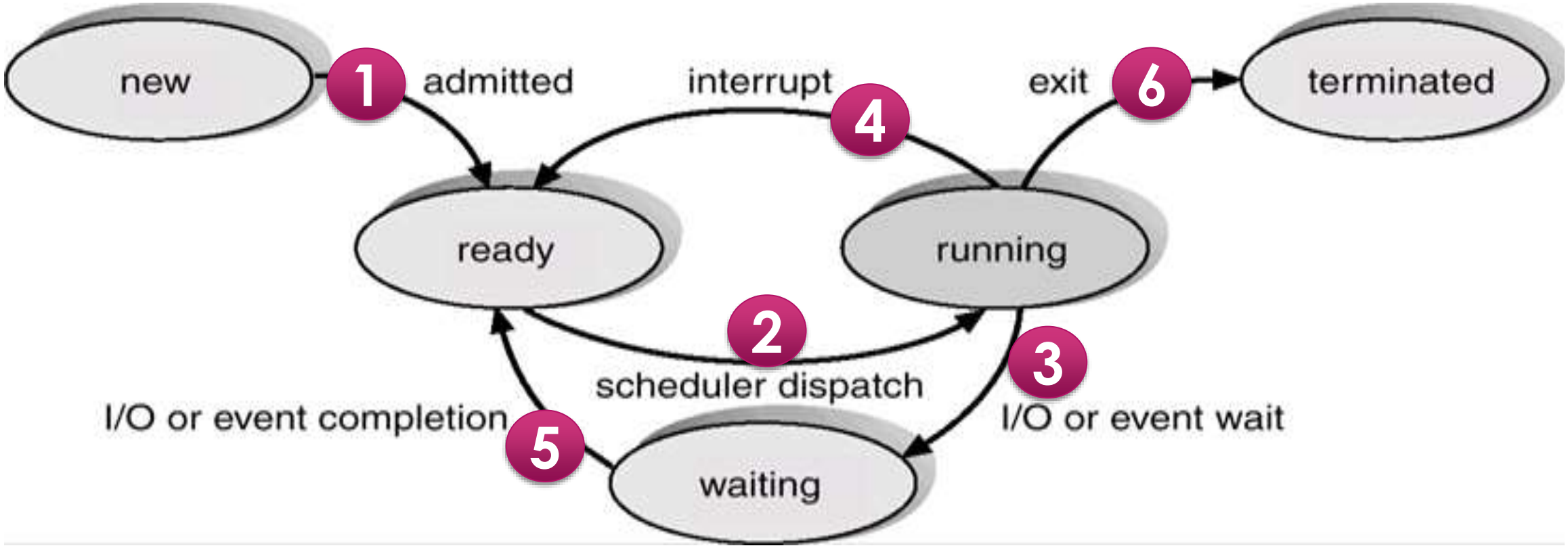
Process Termination…

- Fatal Error (involuntary)
  - Error caused due to a program bug
    - Illegal instruction
    - Referencing non-existent memory
    - Dividing by zero
- Killed by another process (involuntary)
  - A system call telling the OS to kill other process
    - Kill→ UNIX
    - TerminateProcess→Windows

Process State

- As a process executes, it changes ***state***. *The state of a process is defined in part by the current activity of that process.*
- *Each process may be in one of the following states:*
  - **New**: The process is being created.
  - **Ready**: The process is waiting to be assigned to a process.
  - **Running**: Instructions are being executed.
  - **Waiting**: The process is waiting for some event to occur.
  - **Terminated**: The process has finished execution.

Process State



Process state transition

- Transition 1- Occurs when the process is created
  - This state transition is :

    Admitted(Process Name): New → Ready

- Transition 2 - Occurs when the new arrived process to run or all other process have had their share and it is time for the first process to run again
  - This state transition is :

    Dispatch(process Name): Ready →Running

- Transition 3 - Occurs when the process discovers that it can't continue
  - This state transition is:

    Block(process Name): Running →Block

Process state transition…

- Transition 4 – Occurs when the scheduler decides time allotted for the process is expired
  - This state transition is:

    Time-Run-Out(process Name): Running →Ready
- Transition 5 - Occurs when the external event for which a process was waiting happens
  - This state transition is :

    Wakeup(Process Name): Blocked → Ready
- Transition 6 - Occurs when the process has finished execution
  - This state transition is :

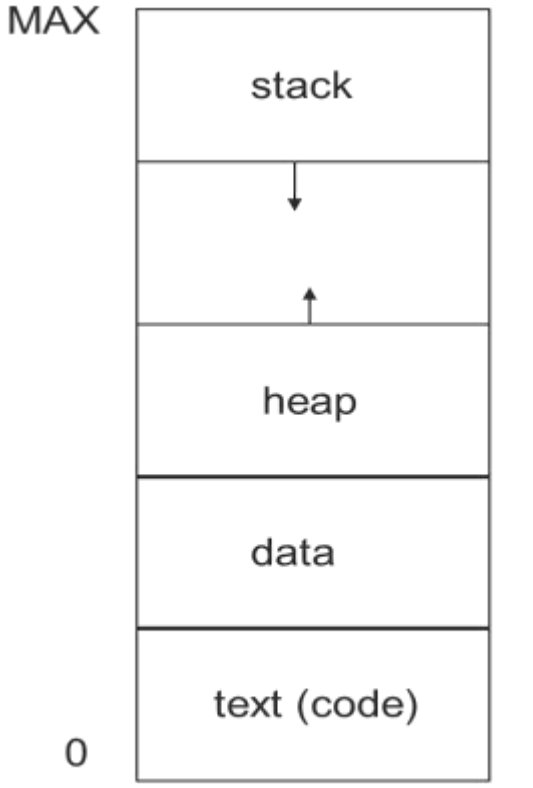
    Exit(Process Name): Running →Terminated

Process management

- Information maintained by OS for process management
  - process context
  - process control block
- OS virtualization of CPU for each process.
  - Context switching
  - Dispatching loop

Implementation…process context

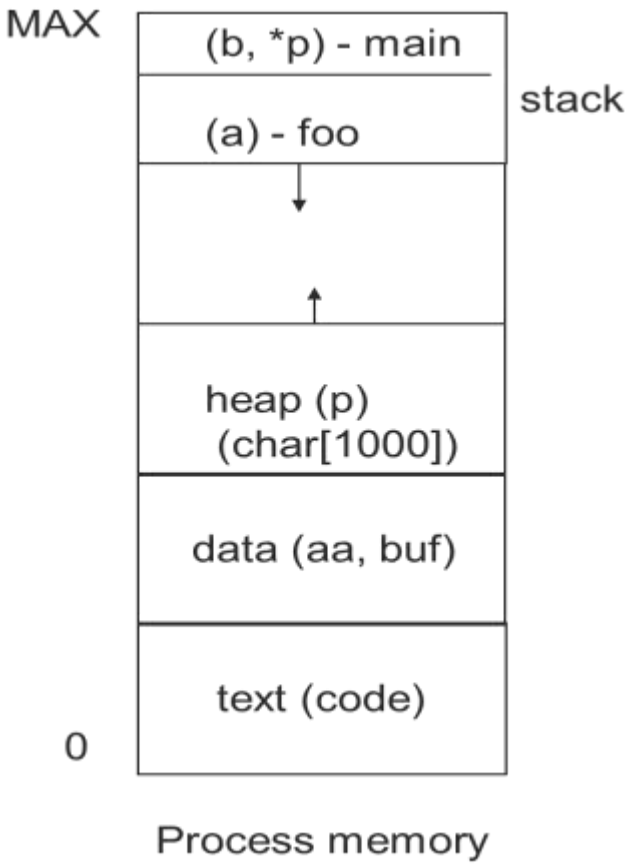
- A Process Context is a run-time information of a particular process at any given time. It's all information needed for OS to restore a suspended process.
- Contains all states necessary to run a program
  - The information the process needs to do the job: code, data, stack, heap.
    - This is known as User level context.



Process in memory

Implementation…process context..

```
…
int aa;
char buf[1000];
void foo() {
  int a;
   …
}
main() {
  int b;
  char *p;
  p = new char[1000];
  foo();
}
```



Implementation…PCB

- A Process Control Block is a group of information about a process for its life time.
- To implement a process model OS maintains a table (array of structures)called the process table also called PCB (process control block)
- One entry for each Process

Process management	Memory management	File management
Registers	Pointer to text segment	Root directory
Program counter	Pointer to data segment	Working directory
Program status word	Pointer to stack segment	File descriptors
Stack pointer		User ID
Process state		Group ID
Priority		
Scheduling parameters		
Process ID		
Parent process		
Process group		
Signals		
Time when process started		
CPU time used		
Children's CPU time		
Time of next alarm		

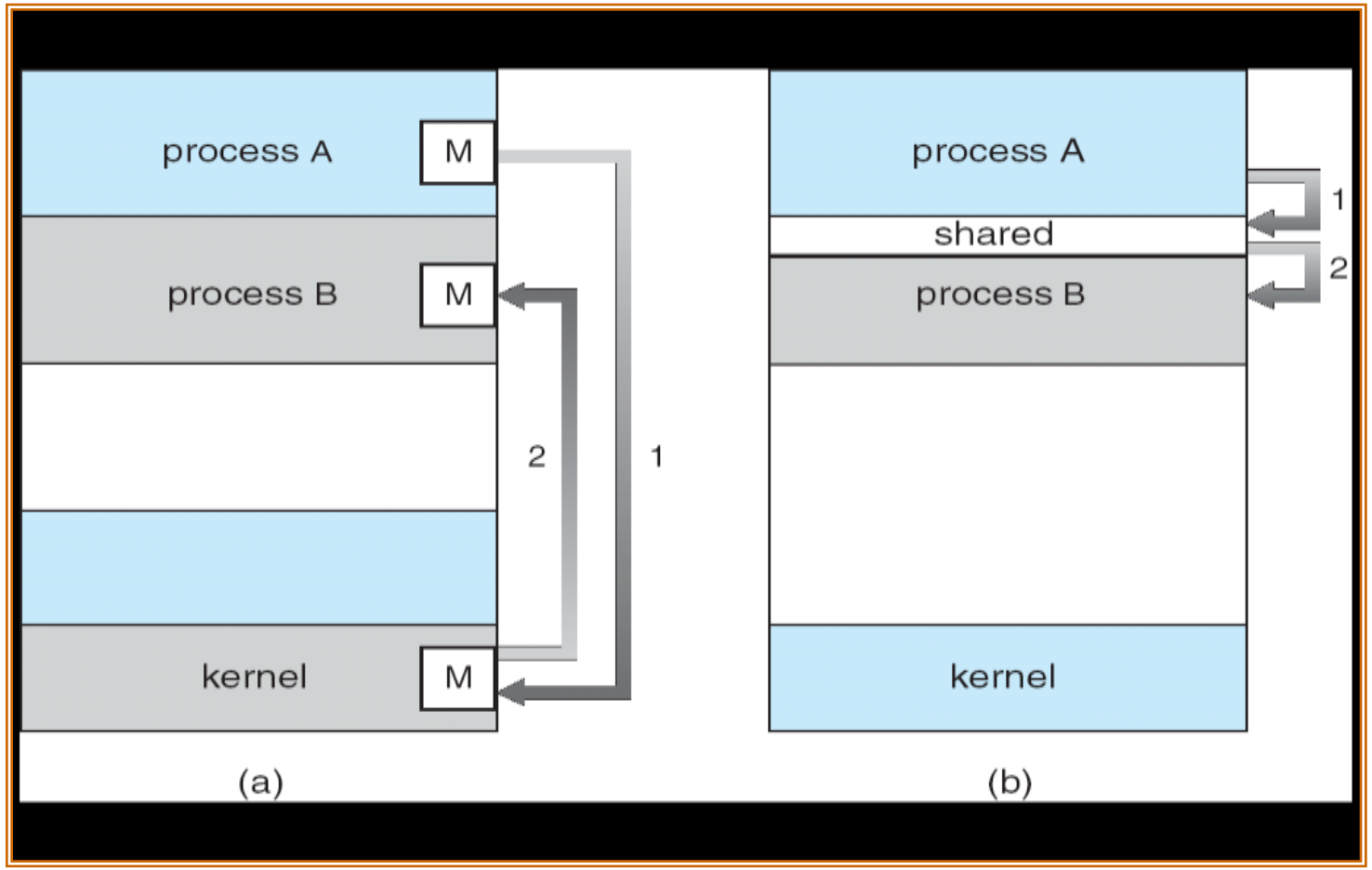
Cooperating Processes

- *Processes executing concurrently in the OS may be either* ***independent*** *or* ***cooperating*** *processes.*
- *Independent* process cannot affect or be affected by the execution of another process.
- *Cooperating* process can affect or be affected by the execution of another process
- Advantages of process cooperation
  - **Information sharing**
  - **Computation speed-up** via parallel sub-tasks
  - **Modularity** by dividing system functions into separate processes
  - **Convenience** - even an individual may want to edit, print and compile in parallel

Cooperating Processes

- Cooperating processes require an inter-process communication (IPC) mechanism to exchange data
  - (a) Message passing
    - Useful for small amounts of data
    - Easier to implement than shared memory
    - Requires system calls and thus intervention of the kernel
  - (b) Shared memory
    - Maximum speed (speed of memory) and convenience
    - System calls required only to establish the shared memory regions; further I/O does not require the kernel

Message passing and shared Memory

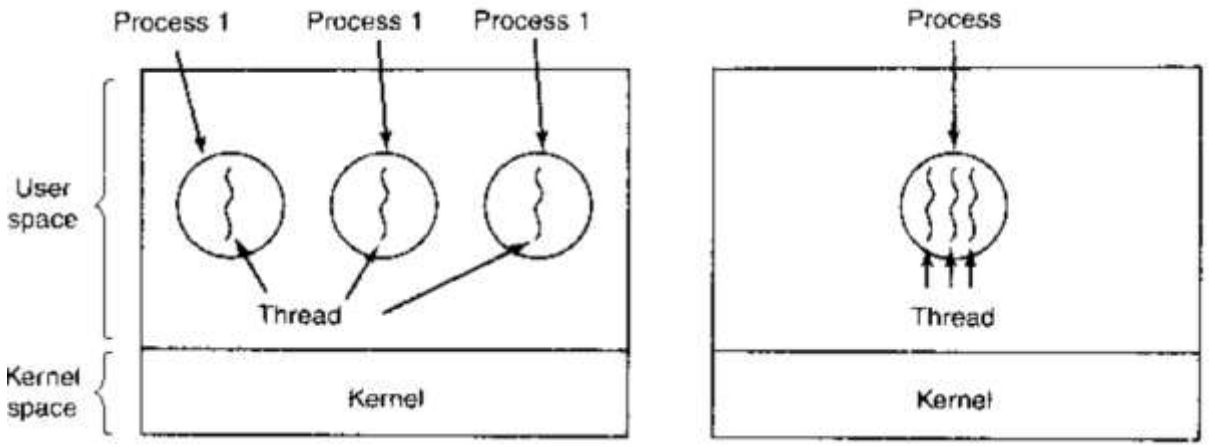


Threads

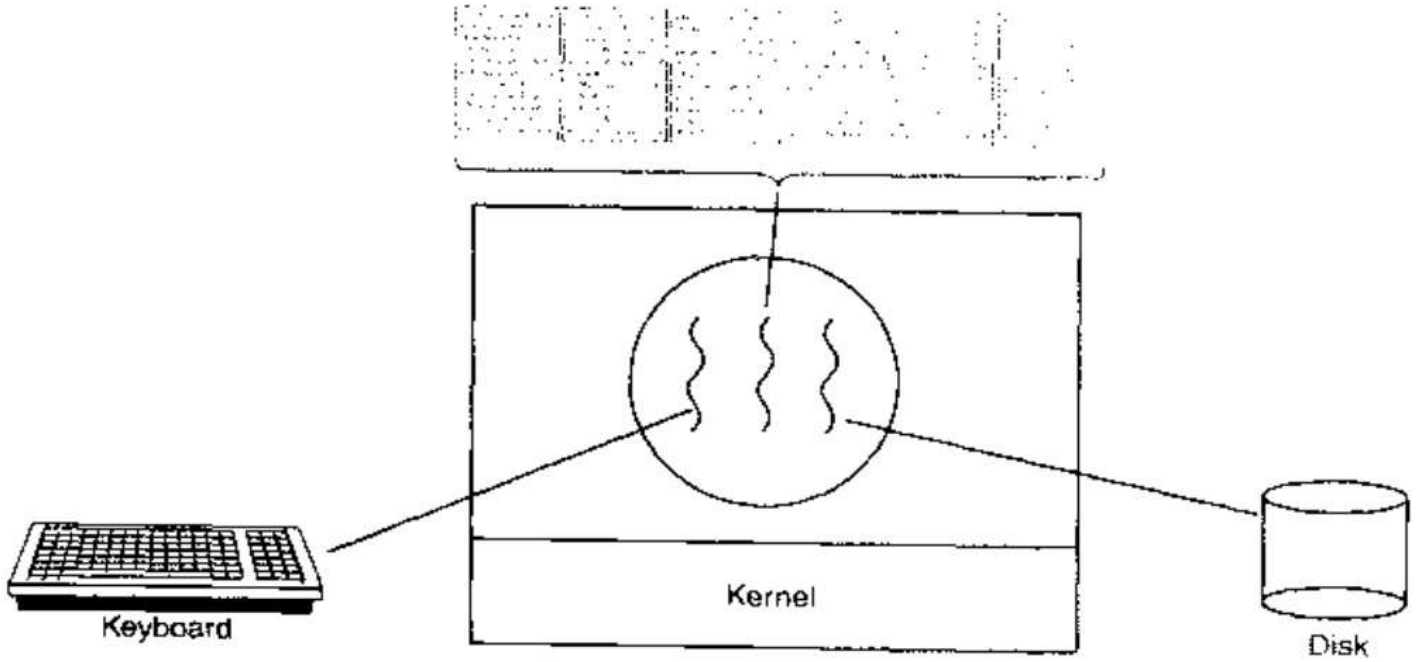
- A thread is a flow of execution through the process code, with its own program counter that keeps track of which instruction to execute next, system registers which hold its current working variables, and a stack which contains the execution history.
- Threads are a way for a program to divide (termed "split") itself into two or more simultaneously (or pseudo-simultaneously) running tasks.
- A thread is also called a **lightweight process**.
- Some examples
  - Threads in Simple Applications
  - Threads in Servers

Threads

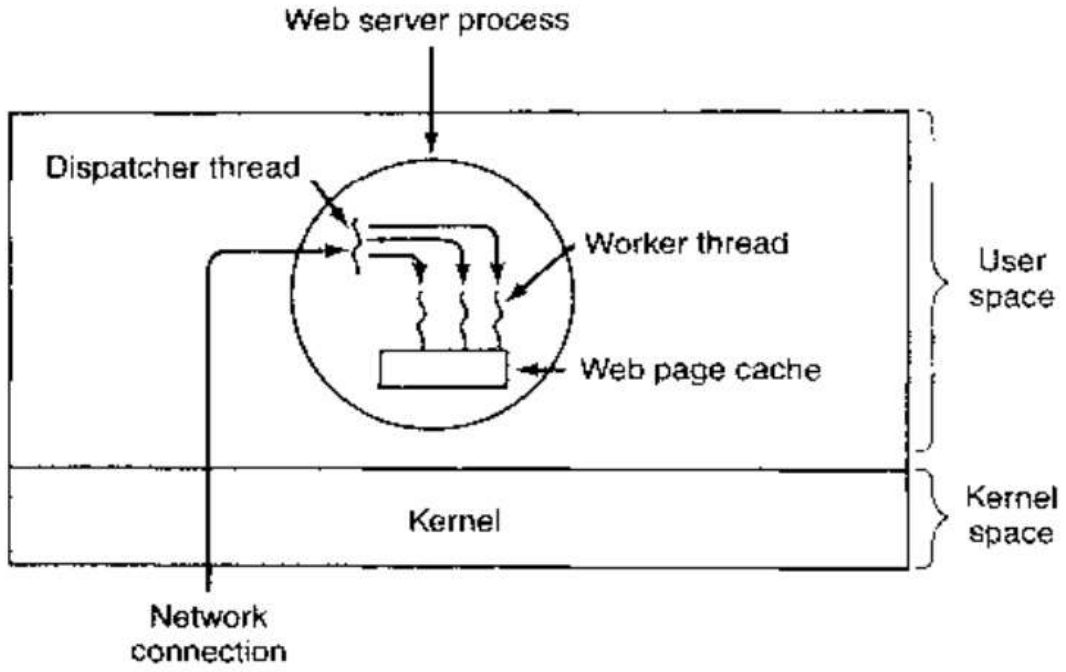
- In traditional OS, each process has a single thread of control
- This was almost the defn. of process
- But most of the time it's desirable to have quasi-parallel



Threads in Simple applications

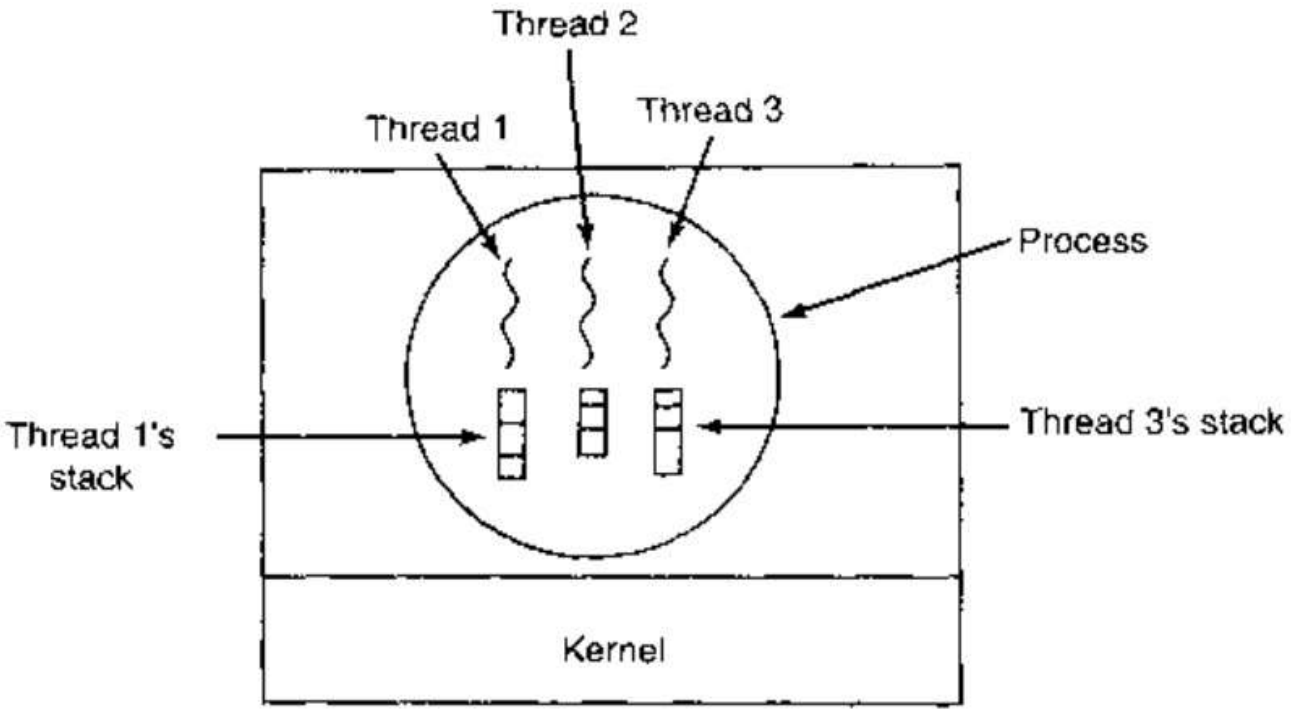


Threads in Server



Thread stack

- Each thread has it's own stack



Process Vs Threads

- Threads operate in the same way as that of processes. Some of the similarities and differences are:
- **Similarities**
  - Like processes threads share CPU and only one thread active (running) at a time.
  - Like processes, threads within a processes execute sequentially.
  - Like processes, thread can create children.
  - And like process, if one thread is blocked, another thread can run.

Process Vs threads

- **Differences**
  - Unlike processes, threads are not independent of one another.
  - Unlike processes, all threads can access every address in the task .
  - Unlike processes, thread are design to assist one other.
  - **Note that**: *processes might or might not assist one another because processes may originate from different users.*

Thread Model

- Threads are entities scheduled for execution on the CPU
- Thread Model add to the process model is to allow multiple execution to take place in the same process environment.
- Thread are considered as a lightweight process

Thread model…

- The process model is based on two independent concepts
  - ***Resource grouping and execution***
- A process has an address space containing text and data…
- Process has thread of execution
- And thread has
  - *PC*
  - *Registers*
  - *stack*

Thread model vs process model

- Threads in a process are not quite as independent as process
- One thread can access every memory address within a process address space
  - *Hence, can read*
  - *Can write*
  - *Even can completely wipeout another thread stack*
  - Hence it's not protected for two reasons
    - *It's impossible*
    - *It should not be necessary*

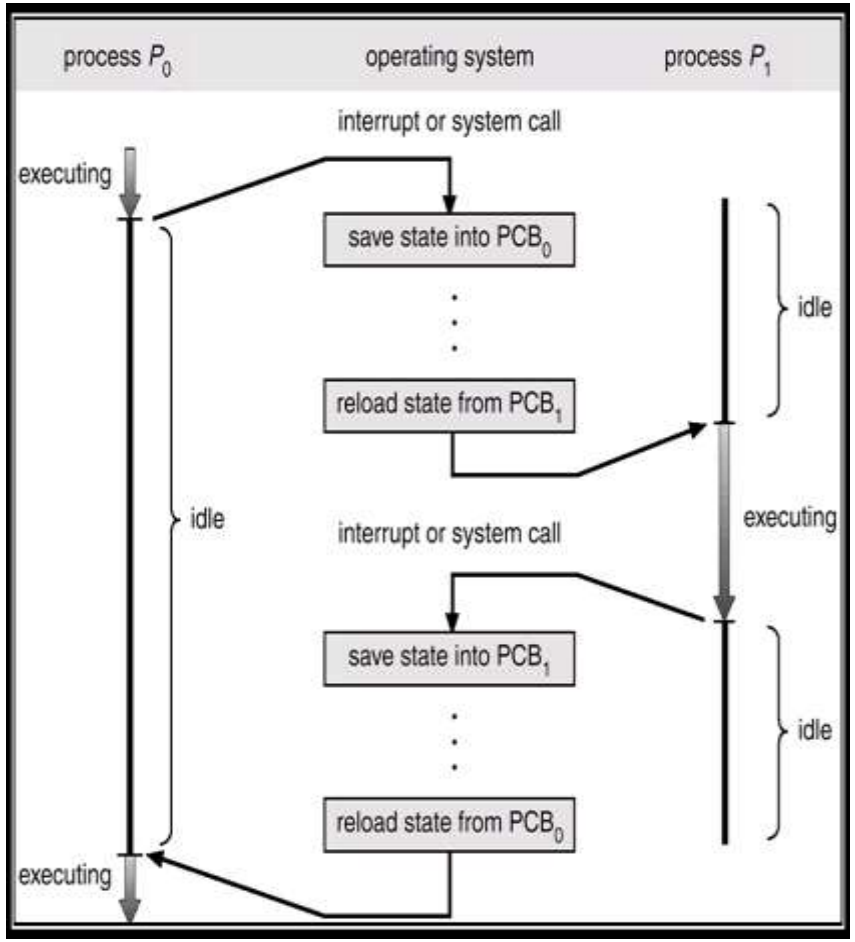
Advantage of threads over Multiple processes

Context Switching

With so little context, it is much faster to switch between threads.

In other words, it is relatively easier for a context switch using threads.

- Time dependent on hardware support. Varies from 1 to 1000 microseconds



Advantage of threads over Multiple processes….

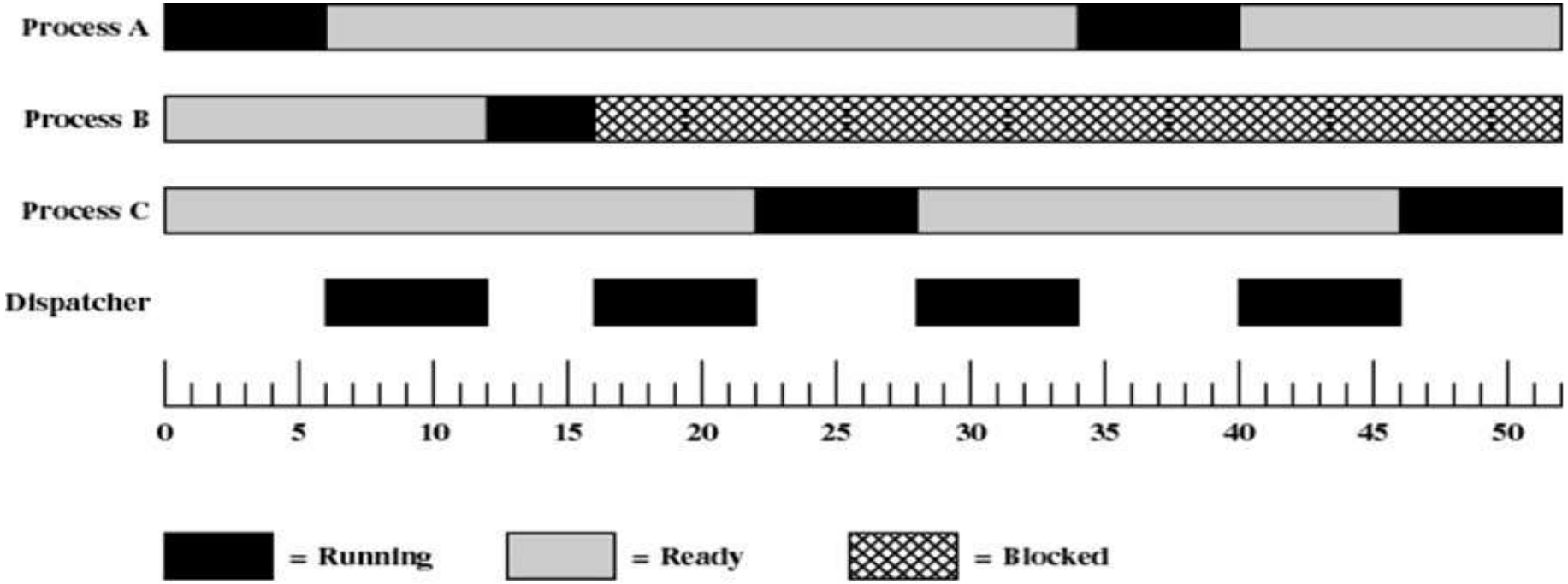
Sharing of resources

- Treads allow the sharing of a lot resources that cannot be shared in process e.g. Sharing code section, data section, open file

Multi-threading	Multi-processing
Address space	Printer
Open files	PC
Global variables	Registers
Child process	Stack
Pending Alarms	state
Signals and signal handlers	
Accounting information	

Disadvantage of threads over Multi-processes

- **Blocking -** The major disadvantage is that if the kernel is single threaded, a system call of one thread will block the whole process and CPU may be idle during the blocking period.



Disadvantage of threads over Multi-processes….

- **Security**
  - Since there is, an extensive sharing among threads there is a potential problem of security.
    - Think through potential deathtrap that comes with resource sharing. Most importantly sharing of memory image/Address space.

Thread state

Just like process a thread can be in one of the following states

- **Ready**
- **Running**
- **Blocked**
- **Terminated**

- Thread transitions are the same with process transition

Thread Usage

1. In many application, multiple activities are going at once

I. Some of this may block from time to time

II. Decomposing such application in to multiple sequential threads that run in quasi-parallel mode.

2.There are no resource attached so they are easy to create and destroy

I. 100 times faster than creating a process.

II. Threads need changes dynamically and rapidly.

Thread usage…

3. Performance gain

I. If not bounded by CPU but for substantial I/O and Computing

4. Useful for systems with multiple CPU and applications that needs to run on multiple CPU and/or needs pseudo parallel

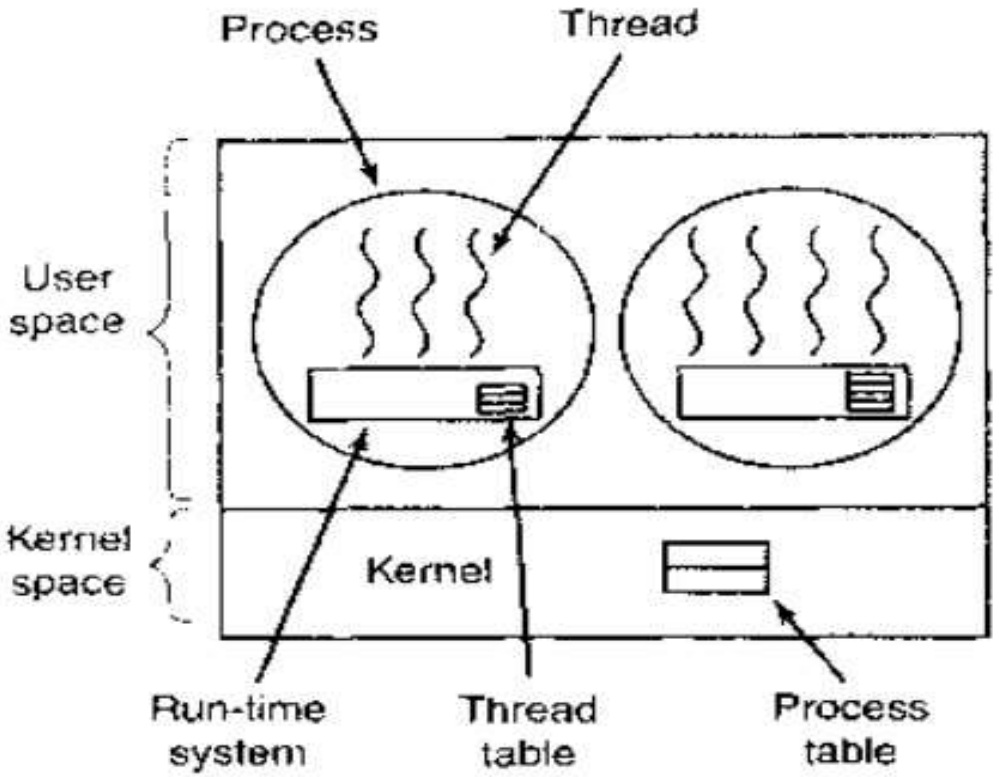
I. Servers
II. Applications like MS Word and MS Excel
III. Mainframe systems

Thread implementation

- Three kinds of thread implementation
  - User level threads
  - Kernel level threads
  - Hybrid implementation

User level threads

- Implement in user-level libraries, rather than via systems calls
- Thread switching does not need to call operating system and to cause interrupt to the kernel.
- In fact, the kernel knows nothing about user-level threads and manages them as if they were single-threaded processes.



User level threads-Advantage

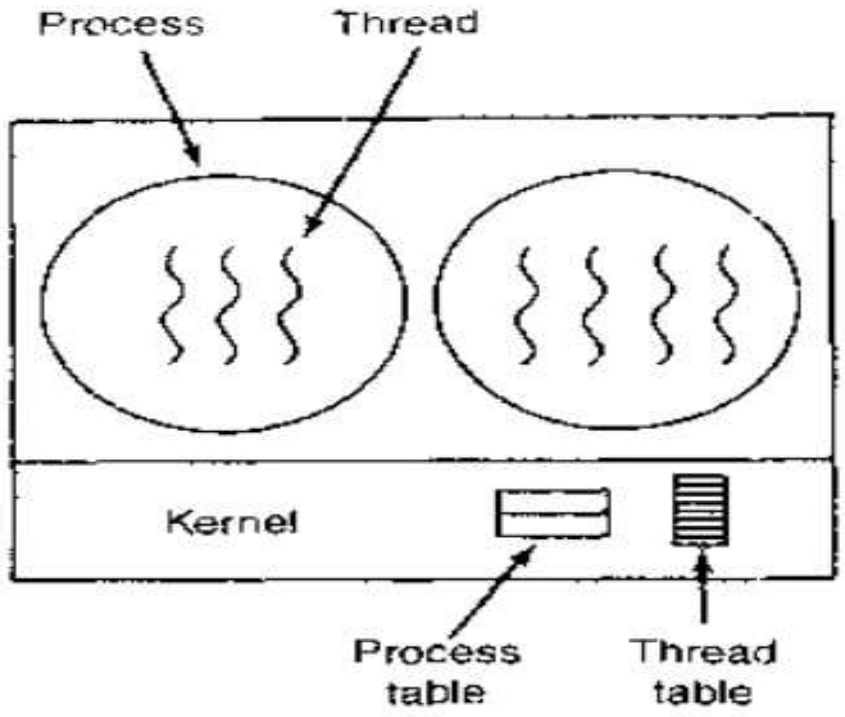
- **Require no modification to operating systems.**
  - Some OS doesn't support thread package implementation
- **Simple Representation**
  - Each thread is represented simply by a PC, registers, stack and a small control block, all stored in the user process address space called thread table.
- **Simple management**
  - creating a thread, switching between threads and synchronization between threads.
  - Help process to have their own customized scheduling.
- **Fast and Efficient**
  - Thread switching is not much more expensive than a procedure call.

User level threads- Disadvantage

- **Lack of coordination b/n threads and OS kernel.**
  - One time slice irrespective of whether process has 1 thread or 1000 threads within.
  - It is up to each thread to relinquish control to other threads.
- **User-level threads requires non-blocking systems call**
  - Ex. Read on call the keyboard before any char is buffered…. Creates changes on the semantics of read
  - Error during a page fault

Kernel-Level Threads

- In this method, the kernel knows about and manages the threads.
- No runtime system is needed in this case. Instead of thread table in each process, the kernel has a thread table that keeps track of all threads in the system.
- In addition, the kernel also maintains the traditional process table to keep track of processes. Operating Systems kernel provides system call to create and manage threads.



The implementation of general structure of kernel-level thread

Kernel-Level Threads- Advantages

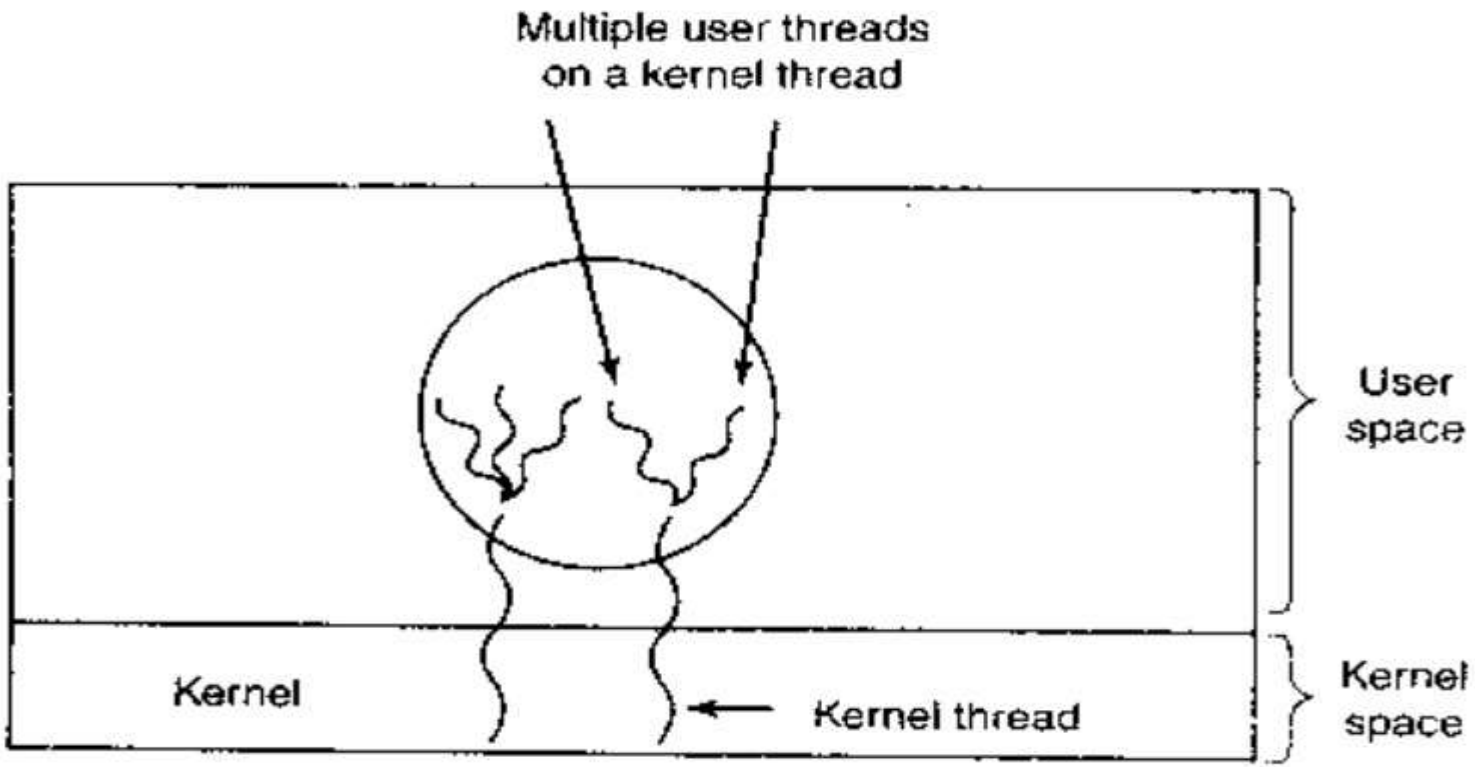
- Because kernel has full knowledge of all threads, Scheduler may decide to give more time to a process having large number of threads than process having small number of threads.
- Kernel-level threads are especially good for applications that frequently block.
  - Kernel level threads do not require any new, non- blocking system calls
- Thread recycling is possible

Kernel-Level Threads- Disadvantages

- **The kernel-level threads are slow and inefficient.**
  - For instance, threads operations are hundreds of times slower than that of user-level threads.
- **It require a full thread control block (TCB) for each thread to maintain information about threads.**
  - As a result there is significant overhead and increased in kernel complexity.

Hybrid implementation

- Combining advantages of user level threads with kernel level threads



End of Chapter 2